Chapter 3. First-class functions

This chapter covers

- Functions for representing computation
- Functions for events and event-driven I/O
- An introduction to higher-order functions
- Variable scope and closures

If you asked a dozen JavaScript programmers what they thought JavaScript got wrong, you'd probably get a dozen different answers. If you asked those same dozen people what JavaScript got *right*, they'd probably all answer, "First-class functions." What does it mean that JavaScript has first-class functions, why does it matter, and how does CoffeeScript make functions even better? All will be answered in good time. First, though, what exactly is a function?

A *function* defines a transformation of input values called *parameters* to an output value called the *return* value. You define a function in CoffeeScript with *literal notation* using an arrow:

```
->
```

That isn't a very useful function, though. It has no parameters and no return value. The function parameters go to the left of the arrow, and the function *body* goes to the right of the arrow:

```
Parameters      Body
  (a, b)    ->  a + b
```

When you *invoke* a function, you provide arguments, and the function transforms them to produce an output value:

```
add = (a, b) -> a + b

add 2, 3
# 5
```